
Data on Fire: A Hands-On Intro to Spark in Fabric

Data Saturday Chicago 2026



Jason Romans

Cloud Data & Integration Developer



The DAX Shepherd



 @sql_jar

 jason-r-sql-jar

 <https://thedaxshepherd.com/>

 Nashville, TN, USA

 Began Career as a SQL Server DBA

 Transitioned to Microsoft BI Stack

 Data Engineering to Data Modeling

Sponsors

Platinum Sponsor



redgate

Gold Sponsor



Semaphor

Silver Sponsor

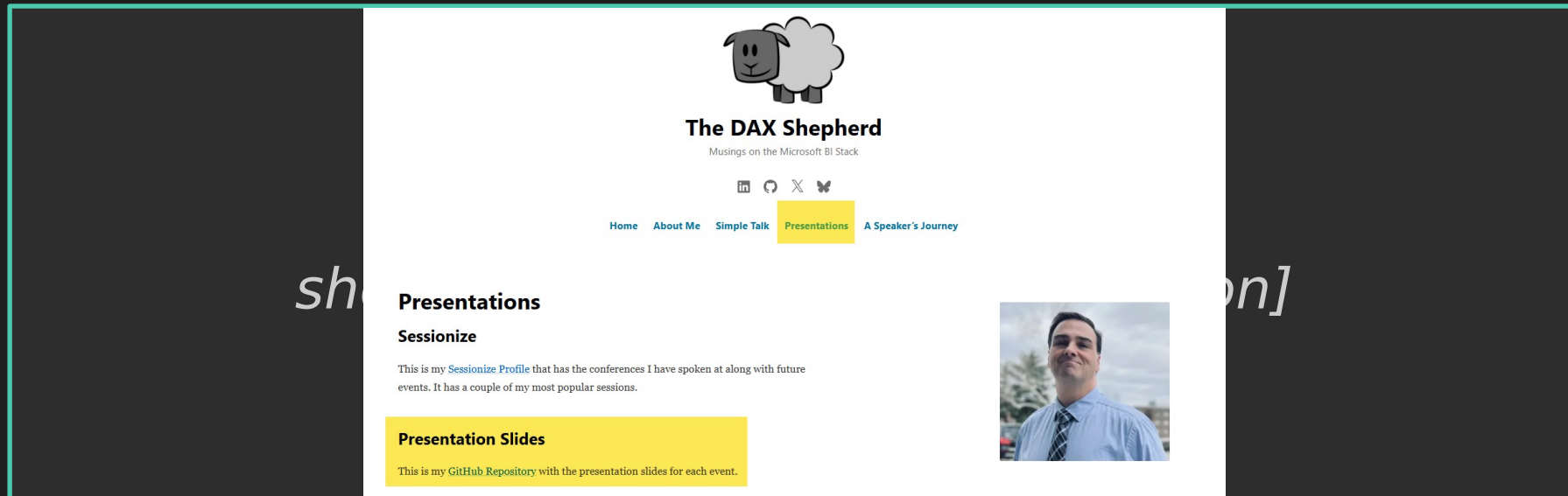


COZYROC

DOWNLOAD THIS PRESENTATION

All slides, resources, and demo files available:

The DAX Shepherd
thedaxshepherd.com/presentations



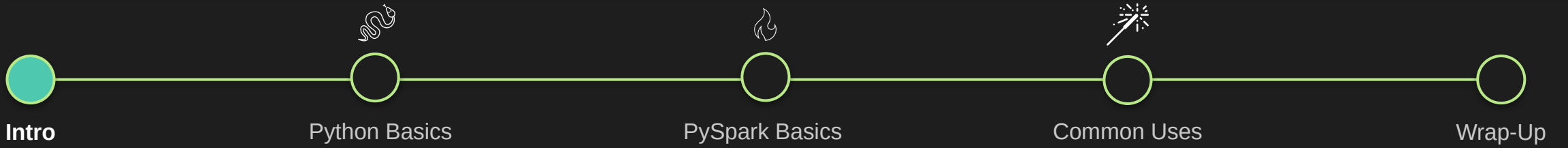
Standing on the Shoulders of Giants



This Photo by Unknown Author is licensed under CC BY-ND

Grateful for the people who share their knowledge, encourage us, answer questions, and help the rest of us learn faster.

YOU ARE HERE



1. Intro

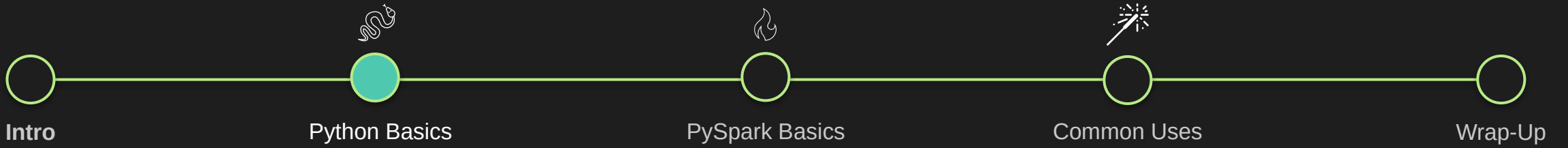
2. Python Basics

3. PySpark Basics

4. Common Uses

5. Wrap-Up

YOU ARE HERE



1. Intro

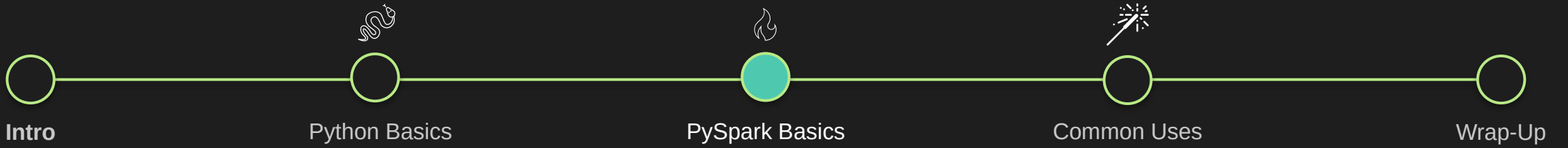
2. Python Basics

3. PySpark Basics

4. Common Uses

5. Wrap-Up

YOU ARE HERE



1. Intro

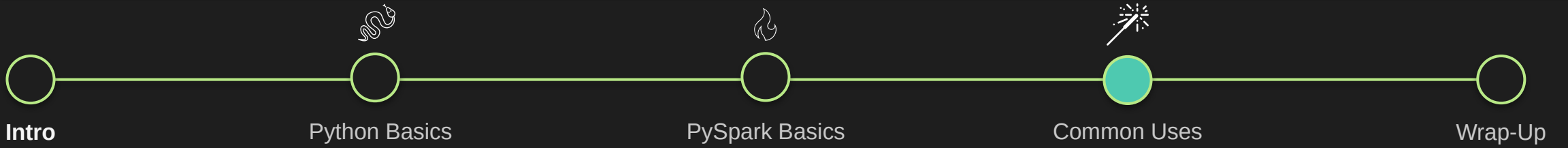
2. Python Basics

3. PySpark Basics

4. Common Uses

5. Wrap-Up

YOU ARE HERE



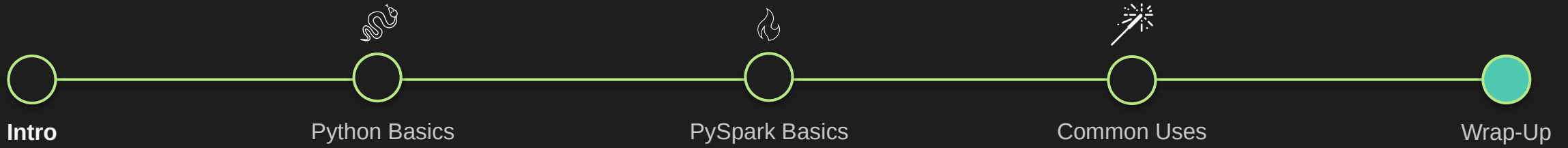
1. Intro

2. Python Basics

3. PySpark Basics

4. Common Uses

5. Wrap-Up



1. Intro

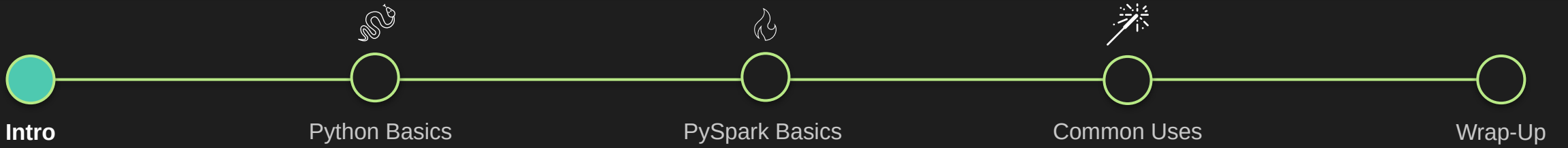
2. Python Basics

3. PySpark Basics

4. Common Uses

5. Wrap-Up

YOU ARE HERE



1. Intro

2. Python Basics

3. PySpark Basics

4. Common Uses

5. Wrap-Up

What lit the fire for Apache Spark

The Netflix Prize

Began Oct 2006

Goal - improve Netflix's
Cinematch algorithm by at least
10%
Prize was 1 million dollars

Took until 2009

Couldn't the Elephant* Help?

* Hadoop's Mascot is an
Elephant

Hadoop was not optimal for
Machine Learning – multiple
passes over disk
Need for new tooling

Shift to in-memory versus disk

Like Analysis Services Multi-
Dimensional to Tabular

Contest must have led to work on
Spark

Flashbacks of submitting homework digitally

Front runner BellKor's Pragmatic Chaos

Merger of teams from AT&A Labs and Commendo Research

July 26, 2009 two teams met minimum requirements

The Ensemble (Spark team) had a better improvement in score

Lost by submitting 20 minutes later

What is Powered by Spark

- Apache Spark
- HDInsight
- Azure Synapse Analytics
- Databricks
- Microsoft Fabric

**Now let
us install
Spark**



Installing Spark

Step 1 of 42

```
$ pip install pyspark
No module named pip:
ModuleNotFoundError
Error importing setuptools module:
Install' command is unavailable until setuptools is
Ensure pip, setuptools, and wheel are up to date
For "upgrade spark pip.ptgspec "ll ergk!
Upgrade pyspark --no-cache-dir setuptools wheel
Value for scheme.headers does not match
to avoid this problem; \f errcode, with exame>
Retrying (Retry(total=4, connect=-4323 after Exceptio
annotate; error: (versygtut), line 230, init _eforl
File '/usr/lib/python3.8/supprocs.py, line 231, in me
trod self.s.connect(sockaddr cannect) timeout
File '/usr/lib/python3.8/socket.py', line 26, i meth
self.s.connect(sockaddr)
Interna to an attack involve'next
Collecting pyspark _apack-3.2.1-bin-hadoopg.3.2.cg int
Downloading Apache-spark-3.2.1-i-nstaller
error Value`for scheme.headers does not match; ae' matc
to avoridel tryh connect be found; > to avoid this problem
Retrying (Retry(total=<connect=,) after Exception annotate
-- again:z https://files.pytonosted.org/poack,org)P/28n-
(nttps:'files.pythonhosted.org(hjaps://packages/5565f.1
confirming) package failed: There was a p: '-crobiem-c t
Exception (too problem confirming the ssl certificate: HTTPS
(host='>>'https://pypi-1.jsom100ht> HTTPSConnectinPool(mo]
Could not install pagages due to an vo space on device
[Errno 28) nò insstall paccages due to an an an OSEserer:
```

**Wait!
Microsoft
Fabric
makes this
easy**



Notebooks in Microsoft Fabric

Can apply to other environments

Notebook Gallery

https://community.fabric.microsoft.com/t5/Notebook-Gallery/bd-p/pbi_notebookgallery

There was a notebook contest (it is closed now):

<https://powerbi.microsoft.com/en-us/blog/introducing-the-first-ever-fabric-notebooks-competition-for-power-bi/>

Microsoft Fabric Architecture

- Data is stored in OneLake
- Lakehouse
 - **Files/** — unmanaged files (CSV, Parquet, JSON, etc.)
 - **Tables/** — managed Delta tables (queryable via SQL endpoint)

ONELAKE

"THE ONEDRIVE
FOR YOUR DATA"

VISUALLY
EXPLAINED

EACH FABRIC ORGANIZATION HAS
ONE DATA LAKE, ENTIRELY MANAGED FOR YOU
IT'S UNIFIED ACROSS ALL REGIONS & YOU PAY
PER GB STORED (NO SCALING NEEDED).

ALL ANALYTICAL ENGINES CAN ACCESS ONELAKE DATA

SQL

SPARK

POWER BI

KQL DB

DATA LAKE + WAREHOUSE
= LAKEHOUSE

WITH FABRIC, YOUR
WAREHOUSE
DATA IS ALSO STORED
IN ONELAKE, AND
ACCESSIBLE IN AN OPEN
FORMAT

THIS IS A
LAKEHOUSE

IT ALLOWS YOU TO DISTRIBUTE
OWNERSHIP TO ORGANIZATIONS,
AND EASILY COLLABORATE ON
DATA

IT CONTAINS



A FILES
AREA



A TABLES
AREA

SHORTCUTS

ALLOW YOU TO QUERY
DATA FROM ONE LAKEHOUSE
TO ANOTHER WITHOUT DATA
MOVEMENT

SHORTCUTS LINK
TO OTHER CLOUDS
TOO!

THE FILES AREA CAN CONTAIN UNSTRUCTURED
& SEMI-STRUCTURED DATA.

IT'S NOT UNCOMMON TO ORGANIZE THEM IN
THREE "AREAS": BRONZE, SILVER & GOLD

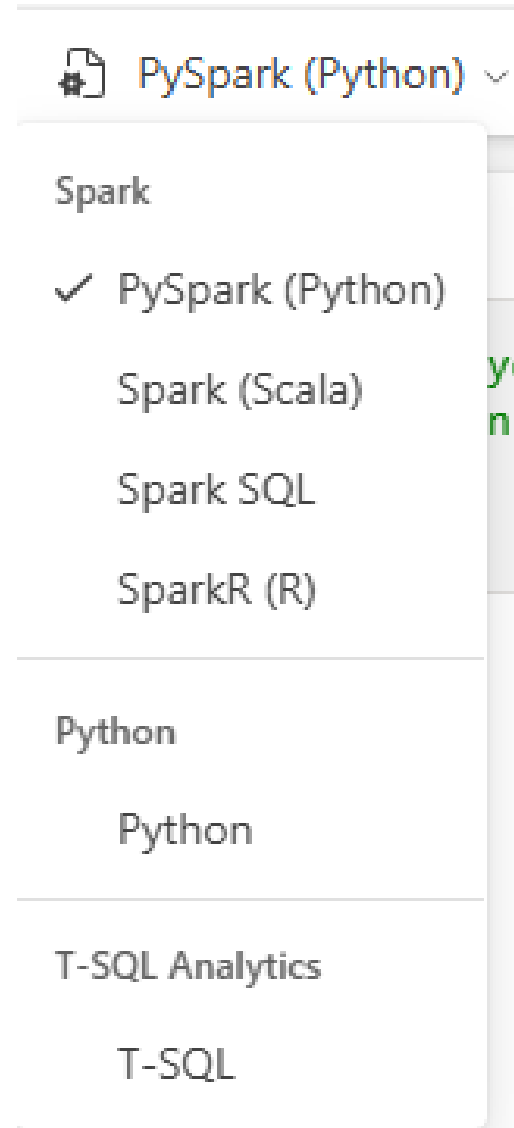
ONELAKE

ADLS GEN2
SDK & APIs
CAN NATIVELY ACCESS ONELAKE

Microsoft Fabric Architecture

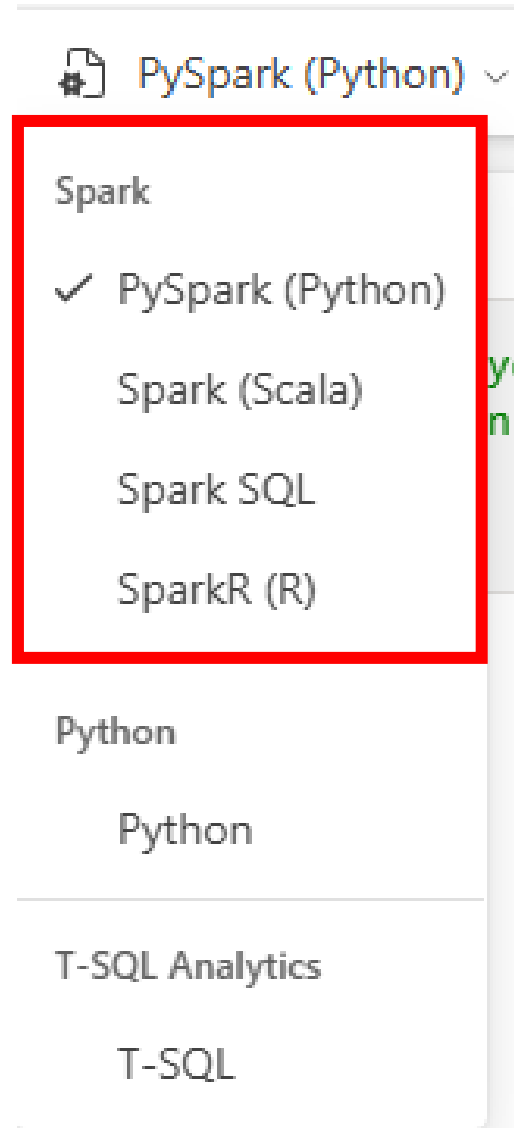
- Compute engines sitting on top of files
- Languages and compute
 - i.e. T-SQL with Warehouse

Compute & Language

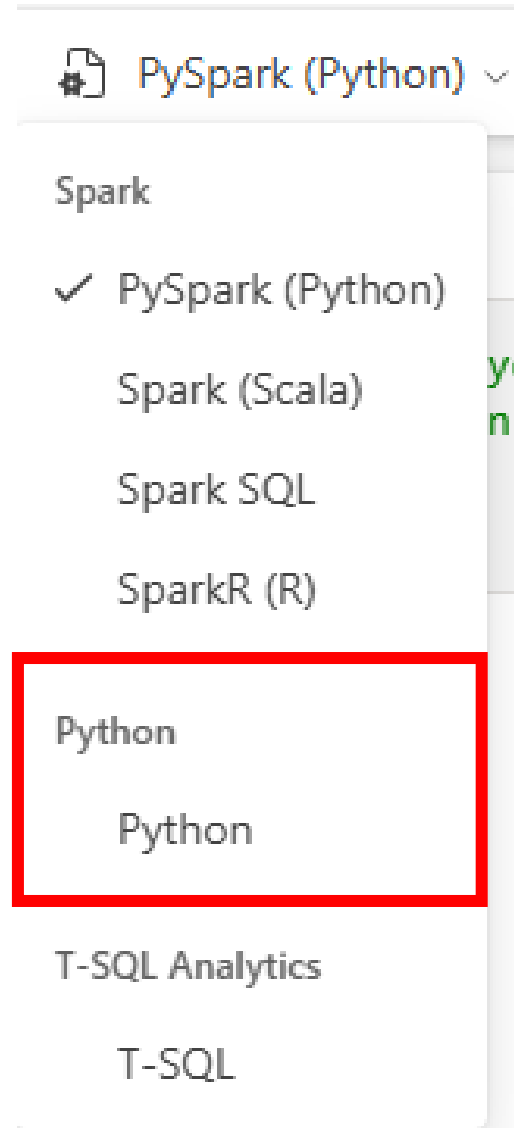


Spark

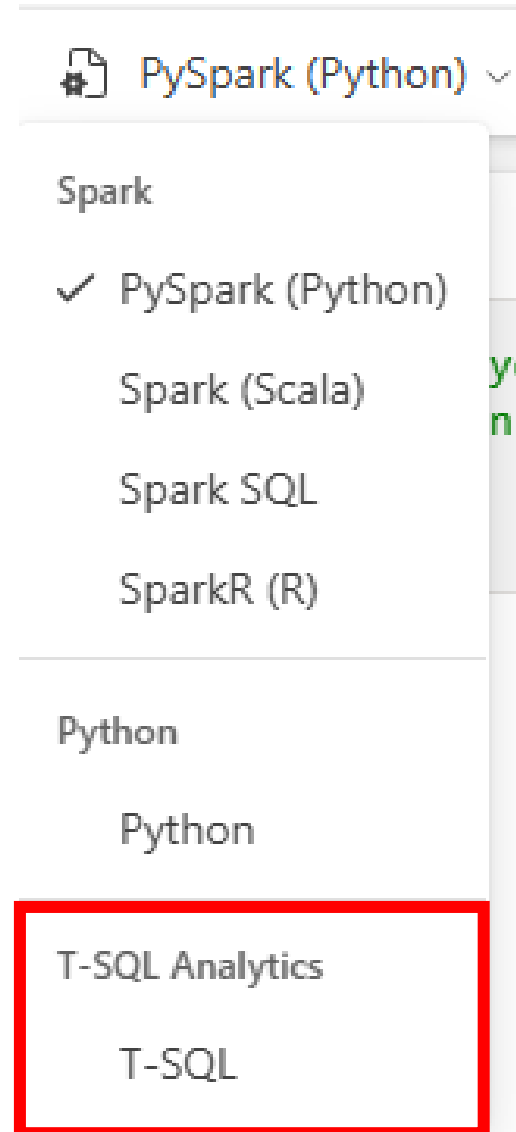
(Python, Scala, SQL, R)



Python (Python)



T-SQL Analytics (T-SQL)



Choosing PySpark or Python Compute (quick)

Scenario	Recommended Notebook
Includes pre-installed DuckDB and Polars libraries	Python Notebooks
Small to medium data (fits in memory)	Python Notebooks (or PySpark on single-node Spark cluster)
Rapid exploration & prototyping	Python Notebooks (or PySpark on single-node Spark cluster)
Large datasets (10GB+) exceeding memory	PySpark Notebooks
Complex data workflows or ETL pipelines	PySpark Notebooks
High-concurrency or parallel execution	PySpark Notebooks
Needs Spark-native APIs (MLlib, SQL, Streaming)	PySpark Notebooks

<https://learn.microsoft.com/en-us/fabric/data-engineering/fabric-notebook-selection-guide>

Choosing PySpark or Python

Scenario	Python Notebooks (2-core VM)	PySpark Notebooks (Spark Compute)
Startup Time	The built-in starter pool initializes in approximately 5 seconds, while the on-demand pool takes around 3 minutes.	Start-up ranges from ~5 seconds (starter pool) to several minutes (on-demand Spark clusters).
Quick Transformations & API Calls	Ideal for small to medium sized datasets (up to 1GB)	Optimized for large datasets using vectorized execution.
Moderate Workloads	Not optimized for data sizes nearing memory saturation	Efficient at scaling via distributed compute.
Handling of Large Datasets	Limited by single-node memory. May struggle with scaling.	Distributed processing ensures scalable handling of multi-GB to TB workloads.
High-Concurrency Execution	Manual FIFO-style parallelism per notebook	System-managed concurrency with support for parallel execution.
Resource Customization & Scaling	Fixed compute (2-core VM); does not auto scale. Users can manually scale out using %%config within the notebook.	Flexible resource allocation; supports autoscaling and custom Spark configurations.

<https://learn.microsoft.com/en-us/fabric/data-engineering/fabric-notebook-selection-guide>

Type of Compute for Notebooks

- Spark Based
 - Cluster
- Single Node Python
 - 2 vCores, 16G RAM
- T-SQL Analytics
 - Warehouse

Python Notebook

- Has libraries installed for dealing with “small-big” data
 - Less than 10 Gigabytes
 - Fits in memory
- Example Libraries installed
 - Polars
 - DuckDB

Languages for Spark

Different choices of languages

Built with Scala

- PySpark (Python)
- Spark (Scala)
- Spark SQL
- SparkR (R)

 PySpark (Python) ▾

Spark

✓ PySpark (Python)

Spark (Scala)

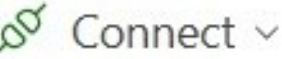
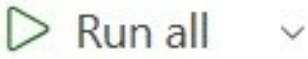
Spark SQL

SparkR (R)

Magic Commands - set language by cell

Magic command	Language	Description
%%pyspark	Python	Execute a Python query against Apache Spark Context.
%%spark	Scala	Execute a Scala query against Apache Spark Context.
%%sql	SparkSQL	Execute a SparkSQL query against Apache Spark Context.
%%html	Html	Execute a HTML query against Apache Spark Context.
%%sparkr	R	Execute a R query against Apache Spark Context.

HomeEditAI toolsRunViewCommentsHistoryDevelopShare



PySpark (Python) ...



»

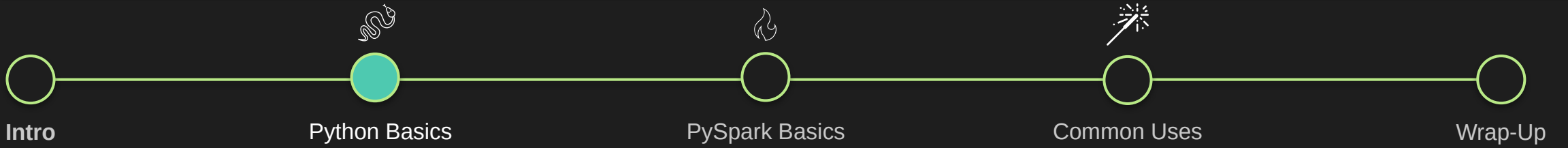
Explorer

Magic Commands

1%%spark

Spark (Scala)

YOU ARE HERE



1. Intro

2. Python Basics

3. PySpark Basics

4. Common Uses

5. Wrap-Up

Python Language



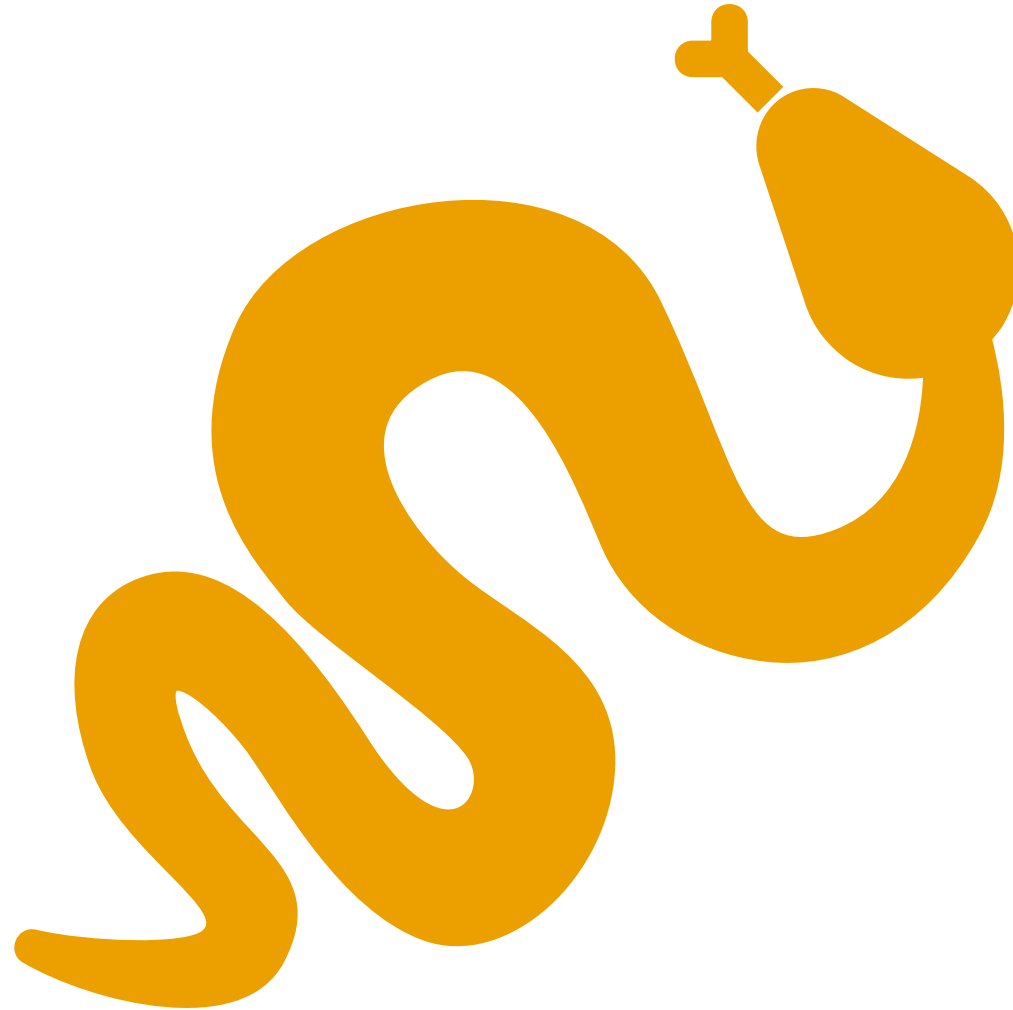
HOW YOU INTERACT
WITH SPARK



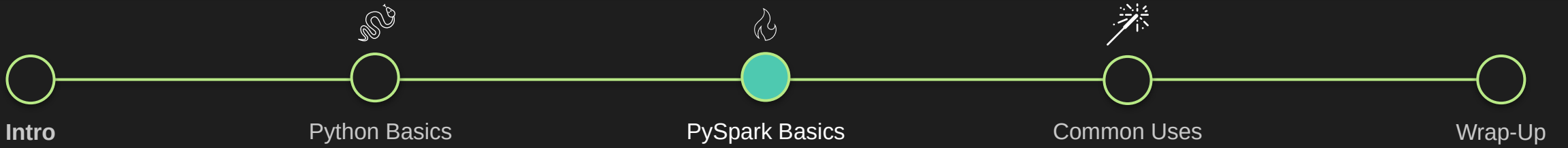
HOW YOU
MANIPULATE THE
DATA

Python Demo

***You can thank me for
not having an actual
picture of a snake here*



YOU ARE HERE



1. Intro

2. Python Basics

3. PySpark Basics

4. Common Uses

5. Wrap-Up

PySpark



Python API for
Spark



Most operations
on a DataFrame



Like Pandas but
distributed

DataFrame

- Resilient Distributed Dataset (RDD)
- Conceptually the same as a table
 - Abstraction
 - Rows
 - Columns

A black and white dog, possibly a pit bull mix, is lying down on a blue and green patterned blanket on a grassy area. The dog is wearing dark sunglasses and has its head resting on the grass. In the background, there are blurred figures of people, suggesting an outdoor event or park setting.

**Spark is
Lazy - in
a good
way**

Lazy Evaluation

- Waits until an action is requested
- Actions
 - Counting number of rows in a Spark DataFrame
 - Showing output
 - Writing data to a file or data source
 - Transferring data from a Spark DataFrame to a native object in Python

Benefits of Lazy Evaluation

- Saves resources
- Plan can be optimized

Pandas (non-spark, historic)
is eager evaluation

Fabric in Visual Studio Code



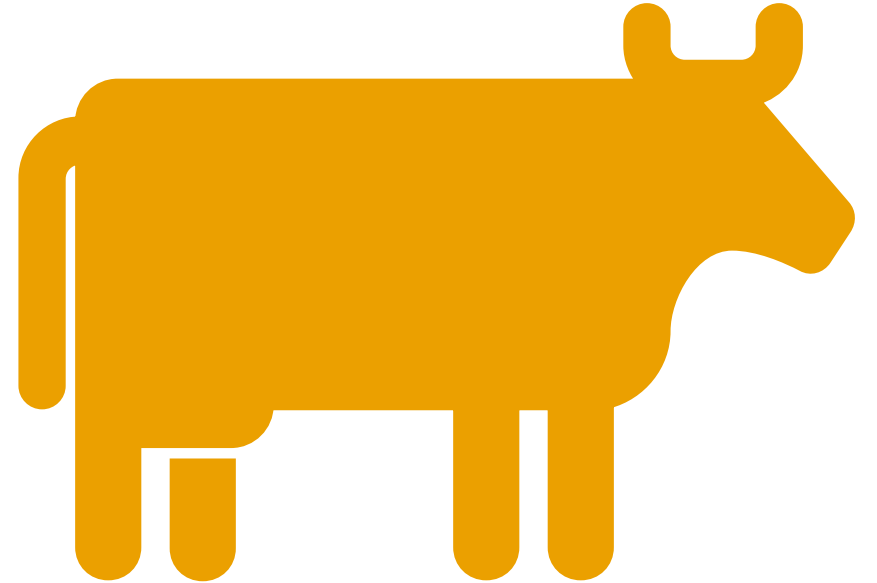
EDIT NOTEBOOKS



CONNECT TO
COMPUTE IN
MICROSOFT FABRIC

Data Wrangler

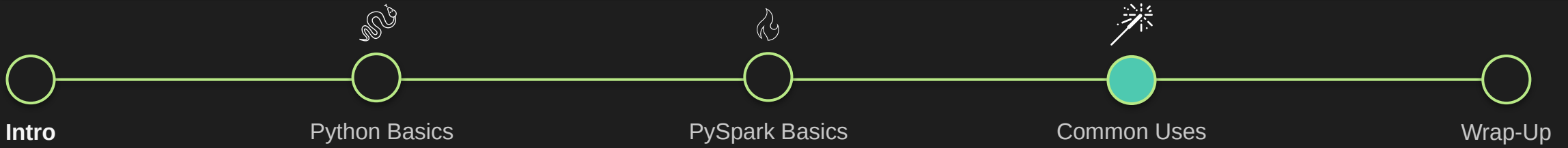
**Like Power Query,
but for PySpark and
Python**



PySpark Demo



YOU ARE HERE



1. Intro

2. Python Basics

3. PySpark Basics

4. Common Uses

5. Wrap-Up

Spark in Practice



Big Data Processing - handling massive datasets



Machine Learning at Scale - training ML models on large data



Real-Time Stream Processing - processing live data streams



ETL & Data Pipelines - building data workflows



Advanced Analytics - complex calculations and BI



Data Exploration & Discovery - interactive data analysis

Two Places to Store Data

- **Files/**

- Raw storage — CSV, Parquet, JSON, etc.
- Not queryable from SQL endpoint
- Good for staging or output files

- **Tables/**

- Managed Delta tables
- Queryable from SQL endpoint and Power BI
- Use `.write.saveAsTable()` to create

Write Modes

- **overwrite** — replaces all existing data
- **append** — adds rows to existing data
- **ignore** — does nothing if data exists
- **error** — throws if data exists (default)
- **Bonus: Delta Merge**
 - Upsert — update if match, insert if not

What Delta Lake Gives You

- **ACID Transactions**

- Writes succeed or fail completely — no partial data

- **Time Travel**

- Query any previous version of the table

- **Merge / Upsert**

- Update existing rows and insert new ones in one operation

Writing to Lakehouse e Demo



Semantic Link & SL Labs

- **Semantic Link (sempy)** — Python library that bridges Spark and Power BI
- Read model data as DataFrames, run DAX queries — no export needed
- **Semantic Link Labs** — extended utilities built on sempy
- Best practice checks, model documentation, and metadata automation

What is a Window Function?

- Performs a calculation across a set of rows related to the current row
- Unlike `groupBy`, original rows are not collapsed — every row is kept
- **The building block:**
`Window.partitionBy().orderBy()`
 - `partitionBy` — defines the group (like GROUP BY)
 - `orderBy` — sort order within each group

The Three Ranking Functions

- **rank()**

- Leaves gaps after ties: 1, 1, 3

- **dense_rank()**

- No gaps: 1, 1, 2

- **row_number()**

- Always unique — ties broken arbitrarily

- *Classic use: filter where rank = 1 to get top row per group*

What Can You Do with Windows?

- **Ranking** — `rank()`, `dense_rank()`, `row_number()`
- **Lag / Lead** — period-over-period comparisons
- **Running Aggregates** — cumulative sum, rolling average
- **Bucketing** — `ntile()` for quartiles and deciles

Resources

Fabric Samples

- <https://github.com/microsoft/fabric-samples>
- Semantic Link
 - <https://learn.microsoft.com/en-us/fabric/data-science/semantic-link-overview>
- Semantic Link Labs
 - <https://github.com/microsoft/semantic-link-labs>

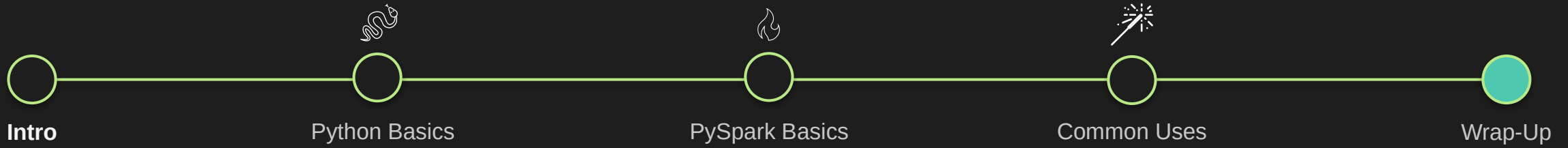
Resources

PySpark Book – Data Analysis with Python and PySpark

- <https://www.oreilly.com/library/view/data-analysis-with/9781617297205/>

PySpark Book – Intro to PySpark (Free HTML version)

- <https://pedropark99.github.io/Introd-pyspark/>



1. Intro

2. Python Basics

3. PySpark Basics

4. Common Uses

5. Wrap-Up

Conclusion

The Spark Has Been Lit

You have the vocabulary: DataFrames, transformations, actions, lazy evaluation. Fabric handles the cluster.

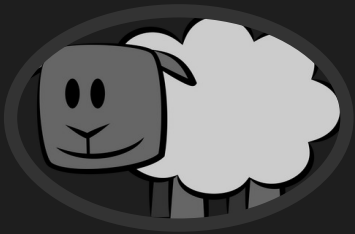
Fabric Clears the Runway

No installation. No setup. Just open a Fabric notebook and start working with data at scale.

Your First Notebook

Awaits Download the session files, pick one use case, and run it. That first cell is all it takes to start.

THANK YOU



The Dax Shepherd

Jason Romans
thedaxshepherd@gmail.com
[linkedin.com/in/jason-r-sql-jar/](https://www.linkedin.com/in/jason-r-sql-jar/)
www.thedaxshepherd.com

Contact Information

